

NinjaFiles Project

ICS202 Project – T251

Salem Badhdah – 202370430

MUSA IBN RASHID – 202437440

1. Introduction

1.1 Project Overview

The NinjaFiles project is a Java-based in-memory file system simulator designed to mimic the core functionality of a Unix/Linux command-line interface (CLI). The primary objective of this project is to implement a hierarchical data structure that efficiently manages files and directories while supporting complex operations such as recursive traversal, path resolution, and string pattern matching.

1.2 Objectives

The main goals of this project are:

- To implement the **Composite Design Pattern** to treat Files and Directories uniformly.
- To utilize **HashMaps** for efficient $O(1)$ average-case access time for file lookups.
- To implement the **Knuth-Morris-Pratt (KMP)** algorithm for efficient string searching within files.
- To manage a hierarchical tree structure with support for relative (`.`, `..`) and absolute paths.

2. System Architecture & Design

2.1 Class Structure

The system is built on Object-Oriented principles, utilizing inheritance and polymorphism.

- **Node (Abstract Class):** The base component of the file system. It holds common attributes such as name and parent directory references. It defines the contract for operations like `getSize()`.
- **Directory:** Extends Node. It acts as a container that can hold other Nodes. It utilizes a `HashMap<String, Node>` to store children, ensuring efficient retrieval.
- **File:** Extends Node. It represents a leaf node containing text content and a size attribute.

- **FileSystem:** The core engine or "Controller" class. It maintains the state of the system (Root and Current Working Directory) and contains the logic for all file operations (mkdir, ls, cd, etc.).
- **NinjaFiles:** The main driver class providing the Command Line Interface (CLI) loop.

3. Data Structures Analysis

3.1. HashMap (Hash Table)

- **Where it is used:**
 - **Class:** `Directory.java`
 - **Variable:** `private HashMap<String, Node> children;`
 - **Methods:** `getChild(String name)`, `addChild(Node child)`, `removeChild(String name)`
- **Why it is used:**

The project specification required efficient access to files. A standard List would require $O(n)$ time to find a file (iterating through every file to check the name). A HashMap provides $O(1)$ (constant time) average complexity for lookups, insertions, and deletions. This makes commands like `cd` and path resolution extremely fast, even in directories with thousands of files.

- **How it is used:**

The directory name (String) is used as the key, and the actual Node object (File or Directory) is the value. When we need to retrieve a specific child, we simply ask the map for the value associated with that name.

Code Snippet:

Location: `Directory.java` (approx. Line 12 and Line 37)

```
// Definition of the data structure
private HashMap<String, Node> children;

// Usage in getChild method: O(1) Lookup
public Node getChild(String name) {
    return children.get(name);
}
```

3.2. Tree (Composite Pattern)

- **Where it is used:**
 - **Classes:** `Node.java` (Abstract), `Directory.java`, `File.java`
 - **Context:** The entire `FileSystem` architecture.
- **Why it is used:** File systems are inherently hierarchical. A Tree structure is the most natural representation, where a `Directory` is a branch node that can contain other nodes, and a `File` is a leaf node. The **Composite Design Pattern** allows us to treat both `Files` and `Directories` as a single type (`Node`), simplifying the code for storage and retrieval.
- **How it is used:** The `FileSystem` class maintains a reference to the root node. Every **Node** keeps a reference to its parent directory (allowing `cd ..` traversal) and **Directory** nodes keep references to their children.

Code Snippet:

Location: `Directory.java` and `File.java`

```
// Directory is a Node
public class Directory extends Node { ... }

// File is also a Node
public class File extends Node { ... }
```

3.3. ArrayList

- **Where it is used:**
 - **Class:** `FileSystem.java`
 - **Methods:** `ls()`, `tree()`, `removeDirectoryRecursive()`
- **Why it is used:** While `HashMap` is great for looking up *one* specific item, it does not maintain order. For commands like `ls` or `tree`, we need to get **all** children and process them (often sorting them alphabetically). `ArrayList` is used to create a temporary, sortable collection of the map's values.
- **How it is used:** We extract the values from the `HashMap` into an `ArrayList`, which allows us to use `Collections.sort()` for alphabetical display.

Code Snippet:

Location: `FileSystem.java` (Inside `ls` method, approx. Line 309)

```
// Convert HashMap values to ArrayList for sorting and
iteration
List<Node> children = new
ArrayList<Node>(targetDir.getChildren());
Collections.sort(children, (a, b) ->
a.getName().compareTo(b.getName()));
```

4. Algorithms Analysis

4.1. Knuth-Morris-Pratt (KMP) Algorithm

- **Where it is used:**
 - **Class:** `FileSystem.java`
 - **Method:** `grep(String pattern, String filePath)`
 - **Helpers:** `buildLPS()`, `kmpSearch()`
- **Why it is used:**

The `grep` command needs to find a string pattern inside a file's content. A naive approach (checking every position) can be slow with a time complexity of $O(N \times M)$. The KMP algorithm improves this to $O(N + M)$ (linear time) by avoiding unnecessary backtracking.

- **How it is used:**
 1. **Preprocessing:** The `buildLPS` method creates an array that indicates how many characters can be skipped upon a mismatch.
 2. **Searching:** The `kmpSearch` method scans the text. If a mismatch occurs, it uses the LPS array to jump ahead rather than restarting the scan from the beginning.

Code Snippet:

Location: `FileSystem.java` (approx. Lines 576-620)

```
◦ // Building the LPS array (Preprocessing)
◦ private int[] buildLPS(String pattern) {
◦     int m = pattern.length();
```

```

    int[] lps = new int[m];
    // ... logic to build lps array
    return lps;
}

// Searching using the LPS array
private boolean kmpSearch(String text, String pattern) {
    // ... efficient scanning logic using lps array to skip
    comparisons
    if (j == m) { return true; } // Pattern found
}

```

4.2. Recursion (Depth-First Traversal)

- **Where it is used:**
 - **Class: Directory.java (getSize) and FileSystem.java (tree, rmRecursive)**
- **Why it is used:** Since the directory depth is unknown (folders inside folders inside folders), iteration is difficult to manage. Recursion allows us to naturally traverse the tree to the deepest "leaf" nodes.
- **How it is used:**
 - **Size Calculation: getSize()** calls itself on all children. The total size is the sum of the current node + the size of all children.
 - **Recursive Removal:** Before removing a directory, the algorithm calls itself on every child to ensure they are deleted first.

Code Snippet:

Location: Directory.java (Method **getSize**, approx. Line 80)

```

@Override
public int getSize() {
    int totalSize = 0;
    // Recursive Step: Sum size of all children
    for (Node child : children.values()) {
        totalSize += child.getSize();
    }
    return totalSize;
}

```

4.3. Path Resolution Parsing

- **Where it is used:**
 - **Class:** `FileSystem.java`
 - **Method:** `resolveNode(String path)`
- **Why it is used:** The system needs to understand user inputs like `../../docs/file.txt`. We cannot simply look up this string in a map. We must parse the path components and navigate the tree step-by-step.
- **How it is used:** The algorithm splits the path string by `/`. It iterates through the array of components. For each component, it checks if it is `.` (stay), `..` (go to parent), or a name (go to child). It updates the current pointer at each step.

Code Snippet:

Location: `FileSystem.java` (Method `resolveNode`, approx. Lines 130-160)

```
String[] components = path.split("/");
for (int i = startIndex; i < components.length; i++) {
    // Handling ".." to go up the tree
    if (component.equals("..")) {
        current = dir.getParent() != null ? dir.getParent() : root;
        continue;
    }
    // Traversing down to a child using HashMap
    Node child = dir.getChild(component);
    current = child;
}
```

5. Supported Commands

The simulator supports the following command set:

Command	Syntax	Description
Make Directory	<code>mkdir [-p] <name></code>	Creates a new directory. The <code>-p</code> flag creates parent directories if they do not exist.
Change Directory	<code>cd <path></code>	Moves the current working directory. Supports <code>.</code> (current), <code>..</code> (parent), and <code>/</code> (root).
List	<code>ls [path]</code>	Lists files and directories. Displays names and file sizes.

Print working directory	pwd	Displays the full absolute path of the current location.
Touch	touch <name> [size]	Creates a file with a specified size. If the file exists, updates it.
Echo	echo "text" > <file>	Writes text content into a file.
Remove	rm <name>	Removes a file or an empty directory.
Recursive Remove	rm -r <dir>	Removes a directory and all its contents recursively.
Tree	tree [path]	Visualizes the directory structure in a tree format.
Disk Usage	du [path]	Calculates the total size of a directory including subfolders.
Grep	grep "pattern" <file>	Searches for a specific text pattern inside a file using KMP.

6. Conclusion

The NinjaFiles project successfully simulates a functional file system. By implementing the Composite pattern, the system handles files and directories with high modularity. The use of HashMaps optimizes file access times, and the implementation of the KMP algorithm ensures efficient text processing. This project demonstrates a strong understanding of data structures (Trees, HashMaps) and algorithmic efficiency (Recursion, KMP).